

Queue Implementation using linked list in C

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

// Define node structure
struct Node
{
    int data;
    struct Node *next;
};

// Front and rear pointers
struct Node *front = NULL;
struct Node *rear = NULL;

// Function declarations
bool isEmpty();
void enqueue();
int dequeue();
void traverse();

int main()
{
    int choice, item;

    while (1)
    {
```

```

printf("\n===== QUEUE MENU =====\n");
printf("1. Enqueue\n");
printf("2. Dequeue\n");
printf("3. Check if Empty\n");
printf("4. Traverse\n");
printf("5. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice)
{
case 1:
    enqueue();
    break;
case 2:
    item = dequeue();
    if (item != -1)
        printf("\nDequeued element: %d\n", item);
    break;
case 3:
    if (isEmpty())
        printf("\nQueue is EMPTY!\n");
    else
        printf("\nQueue is NOT empty.\n");
    break;
case 4:
    traverse();
    break;
case 5:
    printf("\nExiting...\n");
    exit(0);
}

```

```

        default:
            printf("\nInvalid choice! Try again.\n");
        }
    }
    return 0;
}

// Check if queue is empty
bool isEmpty()
{
    return (front == NULL);
}

// Enqueue operation
void enqueue()
{
    int ele;
    printf("\nEnter element to enqueue: ");
    scanf("%d", &ele);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct
Node));
    newNode->data = ele;
    newNode->next = NULL;

    if (isEmpty())
    {
        front = rear = newNode;
    }
    else
    {

```

```

        rear->next = newNode;
        rear = newNode;
    }

    printf("Element enqueued successfully!\n");
}

// Dequeue operation
int dequeue()
{
    if (isEmpty())
    {
        printf("\nQueue Underflow (Empty Queue)\n");
        return -1;
    }

    struct Node *temp = front;
    int val = temp->data;

    front = front->next;
    if (front == NULL) // if queue becomes empty after
deletion
        rear = NULL;

    free(temp);
    return val;
}

// Traverse queue
void traverse()
{

```

```
if (isEmpty())
{
    printf("\nQueue is empty.\n");
    return;
}

struct Node *temp = front;
printf("\nQueue elements: ");
while (temp != NULL)
{
    printf("%d ", temp->data);
    temp = temp->next;
}
printf("\n");
}
```

Output :

```
===== QUEUE MENU =====
1. Enqueue
2. Dequeue
3. Check if Empty
4. Traverse
5. Exit
Enter your choice: 1

Enter element to enqueue: 10
Element enqueued successfully!

===== QUEUE MENU =====
1. Enqueue
2. Dequeue
3. Check if Empty
4. Traverse
5. Exit
Enter your choice: 1

Enter element to enqueue: 20
Element enqueued successfully!

===== QUEUE MENU =====
1. Enqueue
2. Dequeue
3. Check if Empty
4. Traverse
5. Exit
```

Enter your choice: 4

Queue elements: 10 20

```
===== QUEUE MENU =====
1. Enqueue
2. Dequeue
3. Check if Empty
4. Traverse
5. Exit
Enter your choice: 3
```

Queue is NOT empty.

```
===== QUEUE MENU =====
1. Enqueue
2. Dequeue
3. Check if Empty
4. Traverse
5. Exit
Enter your choice: 2
```

Dequeued element: 10

```
===== QUEUE MENU =====
1. Enqueue
2. Dequeue
3. Check if Empty
4. Traverse
```

```
5. Exit
Enter your choice: 4
```

Queue elements: 20

```
===== QUEUE MENU =====
1. Enqueue
2. Dequeue
3. Check if Empty
4. Traverse
5. Exit
Enter your choice: 2
```

Dequeued element: 20

```
===== QUEUE MENU =====
1. Enqueue
2. Dequeue
3. Check if Empty
4. Traverse
5. Exit
Enter your choice: 5
```

Exiting...